05 递归和广义表

- 递归
- 递归转化为非递归
- 广义表

05 递归和广义表

- 递归
- 递归转化为非递归
- 广义表

递归的概念

- **能采用**递归解决的问题通常具有这样的特征
  - 规模为**N的**问题可以被分解为一些规模较小的问题，**而且**这些规**模**较小的问题的求解方法与原来问题的求解方法是一样的，**然后从**这些小问题的解可以方便地构造出大问题的解(**分而治之的策略，即分治法**)
  - 这些小问题可以用同样的方式继续分解成规模更小的问题，当问题分解到足够小的时候，例如**N=1，能直接得到解**

- 递归过程会调用其自身
  - **可以用**递归解决的问题是问题本身的定义是递归的
  - **或者其中的主要数据**结构是递归定义的

定义是递归的

$$n! = \begin{cases} 1, & \text{当} n = 0 \text{时} \\ n * (n-1)!, & \text{当} n \geq 1 \text{时} \end{cases}$$

- 阶乘函数的递归算法

```
long Factorial(long n) {
    if (n == 0)  return 1;          ← 递归结束条件
    else  return n * Factorial(n-1); ← 递归步骤
}
```

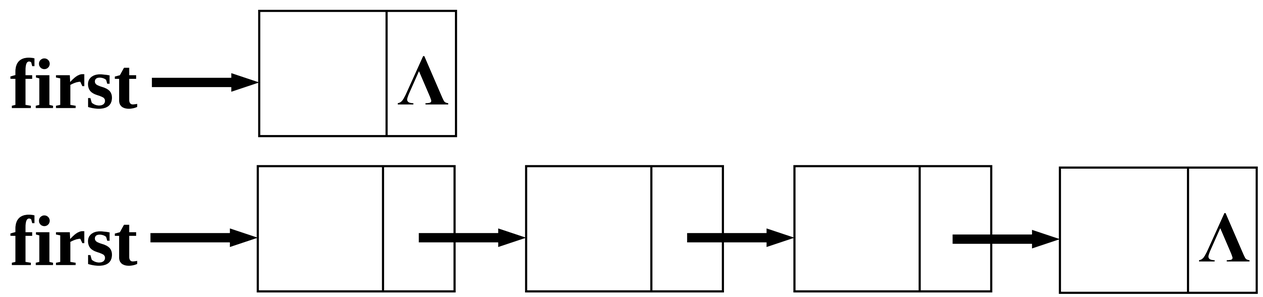
- 递归过程可看作两个过程的组合
  - 第一个过程是大问题逐渐被**分解**成小问题，直到递归终止条件
  - 第二个过程则是从小问题的解**合成**出大问题的解

求解阶乘n!的过程


主程序 main : fact(4)
参数 4　计算 4*fact(3)　返回 24
参数 3　计算 3*fact(2)　返回 6
参数 2　计算 2*fact(1)　返回 2
参数 1　计算 1*fact(0)　返回 1
参数 0　直接定值 = 1　返回 1
递归调用
参数传递
结果返回
回归求值

数据结构是递归的

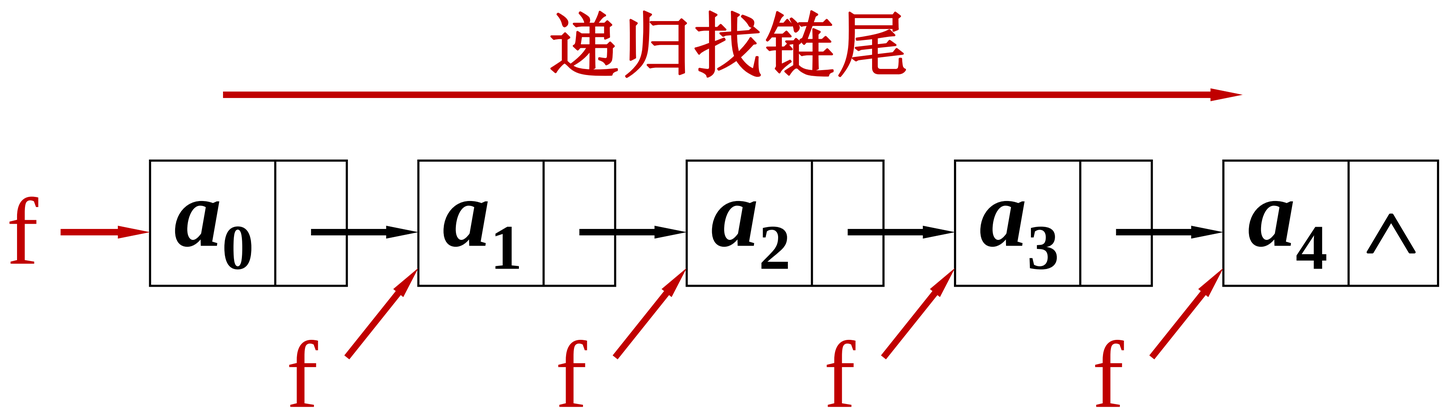
- 很多数据结构都可以用递归的方式描述



  - 单链表
    - 只有一个结点的单链表，是一个单链表
    - 长度为n的单链表，是由链表的第一个结点，其指针指向由剩余n-1个结点形成的单链表
  - 栈
    - 深度为0的栈则是空栈
    - 深度为n的栈，是由一个深度为n-1的栈和当前栈顶元素构成

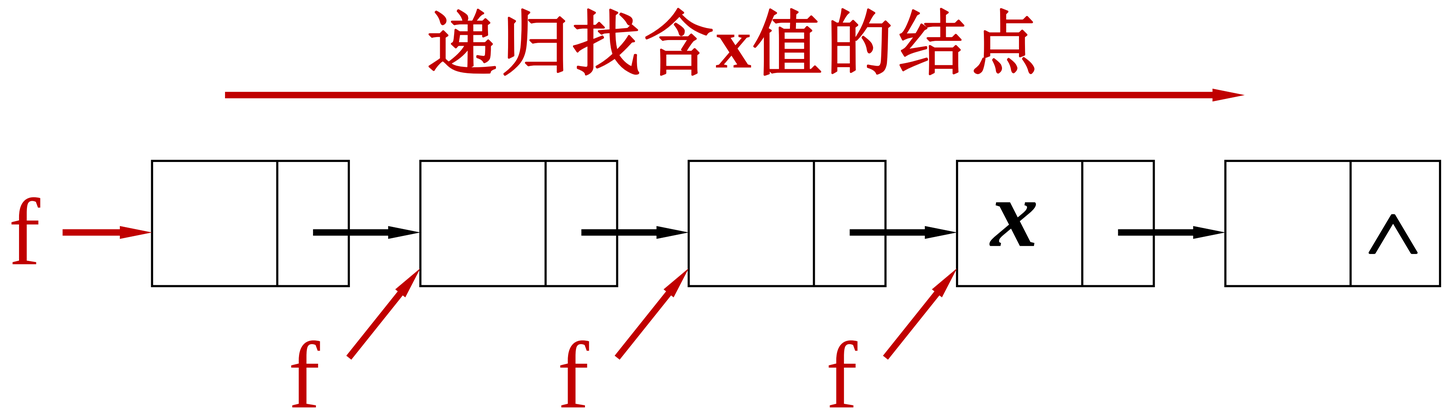
在链表中搜索最后一个结点

```
template <class Type> void Print(ListNode <Type> *f) {
    if (f->link==NULL)
        cout<<f->data<<endl;
    else Print(f->link);
}
```



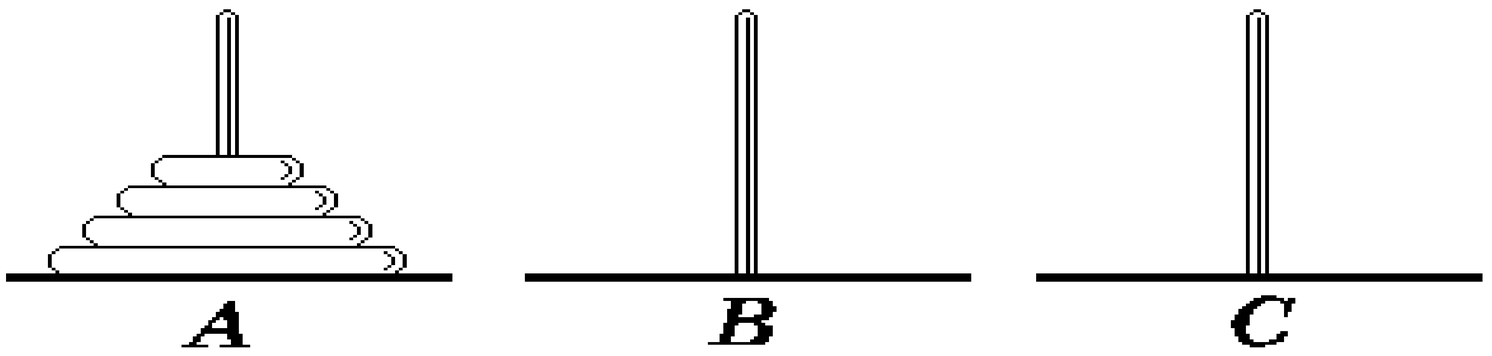
在链表中寻找等于给定值的结点

```
template <class Type> void Print(ListNode <Type> *f, Type &x) {
    if (f != NULL)
        if (f->data == x)
            cout<<f->data<<endl;
        else Print(f->link, x);
}
```

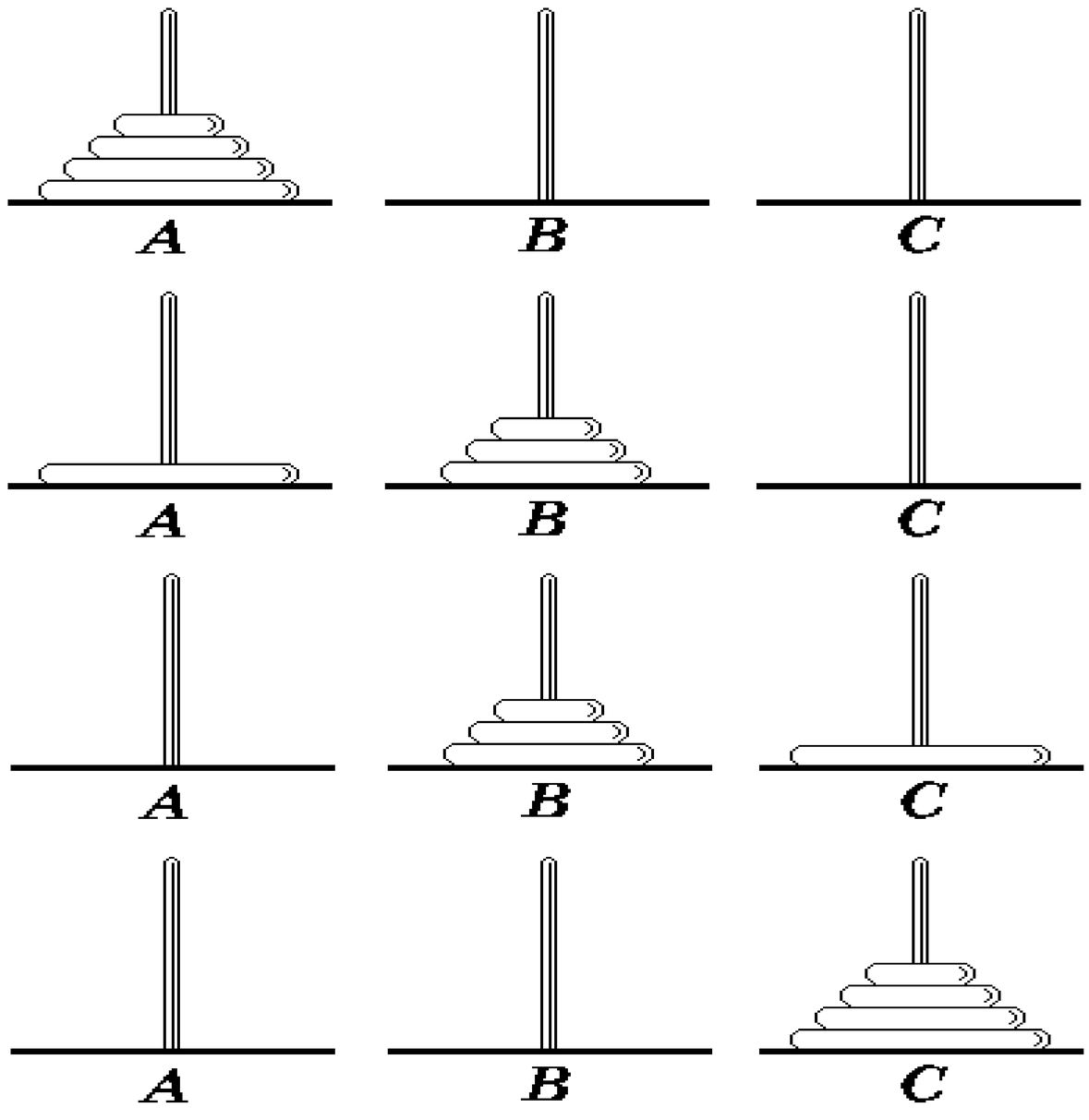


问题的解法是递归的

- **Hanoi塔**问题
  - **有3根针和若干大小不等的金片**
  - **初始时，金片全在A针上，按从小到大顺序叠放，最小的在最上面，最大的在最下面**
  - **要求把这些金片全部移到C针上，在移动的过程中可以利用B针，但不论任何时候在每根针上的金片都必须保持小的金片放在大的金片上，并且每次只能把一个金片移到另一个金片上**



Hanoi塔问题的递归解法

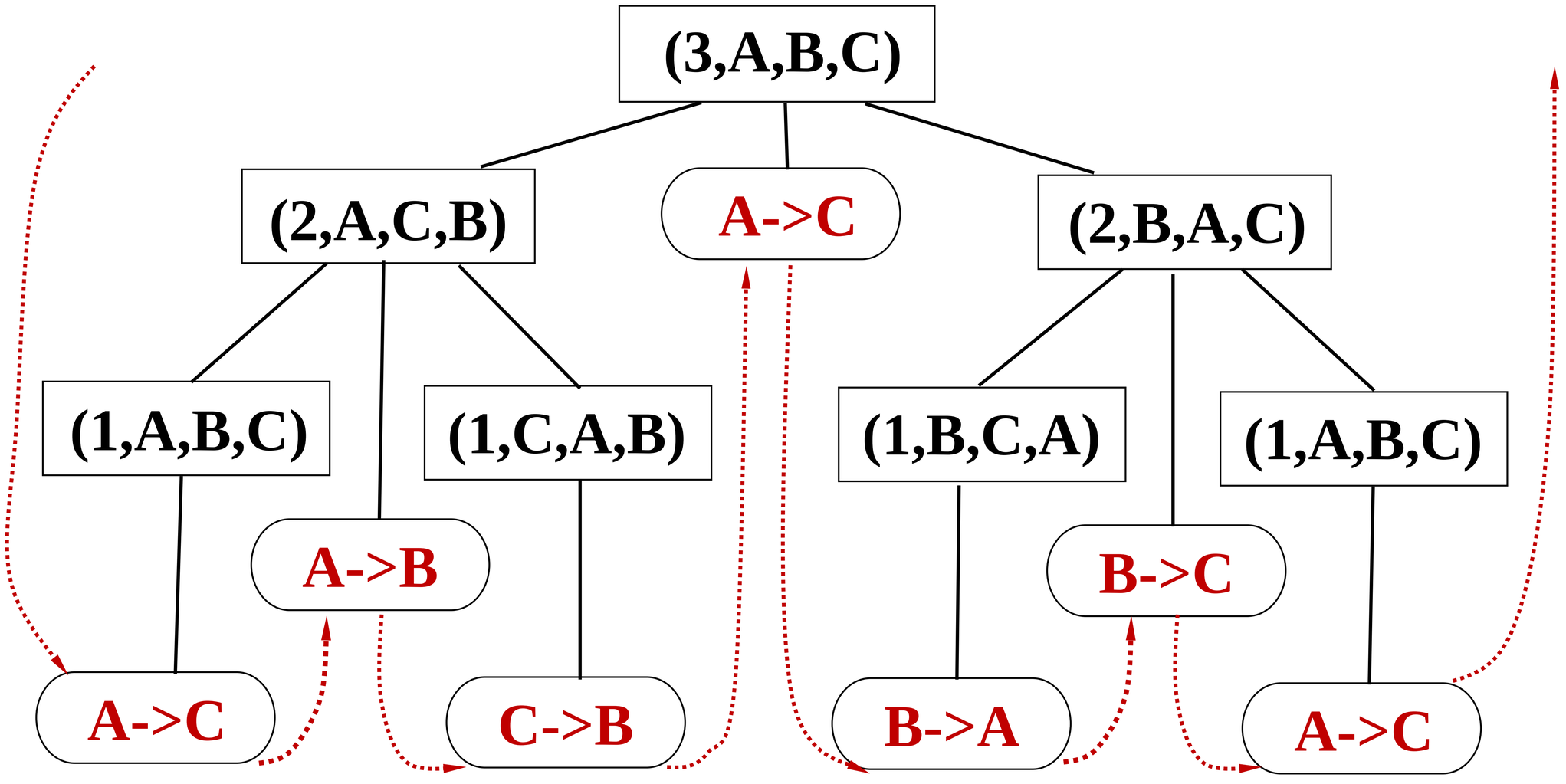


Hanoi塔问题的递归解法

- 如果n=1，则将这一个盘子直接从A柱移到C柱上
- 否则，执行以下三步
  - ① 用C柱做过渡，将A柱上的n-1个盘子移到B柱上(即递归调用)
  - ② 将A柱上最后一个盘子直接移到C柱上(即n=1，可直接求解)
  - ③ 用A柱做过渡，将B柱上的n-1个盘子移到C柱上(即递归调用)

```
void Hanoi(int n, String A, String B, String C) {
    if (n==1)  cout<<“move”<<A<<“to  ”<<C<<endl;
    else {
        Hanoi(n-1, A, C, B);
        cout<<“move ”<<A<<“to ”<<C<<endl;
        Hanoi(n-1, B, A, C);
    }
}
```

Hanoi塔问题的递归解法



递归程序的时间复杂度分析

- **以Hanoi塔问题为例**
- **当n = 0时，T(n) = 0**
- **当n > 0时，T(n) = 2T(n-1) + 1**

- **T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 = …**

  $= 2^{n-1} + 2^{n-2} + \ldots + 1 = 2^n - 1$

递归程序的正确性证明

- 递归程序在逻辑上与常见的数学归纳法类似，因此往往可以用数学归纳法来证明递归程序的正确性
- 在证明时，需要证明两个部分
  - 一个是递归的基础，即作为递归终止状态的最基础情况的正确性，例如在Hanoi塔问题中n=1时的正确性
  - 按照数学归纳法，设n-1规模是正确的，证明n规模的正确性，例如假设在n-1的情况下Hanoi塔问题的解法是正确的，则由于n规模的Hanoi塔问题是通过n-1规模的解法解决的，这样就可以证明Hanoi塔问题的递归程序的正确性

构成递归的条件

- 不能无限制地调用本身，必须有一个出口，化简为非递归状况直接处理

```
Procedure <name> (<parameter list>) {
    if (<initial condition>)
        return (initial value);
    else
        return (<name> (parameter exchange));
}
```

递归与回溯

- 对一个包含许多结点，且每个结点有多个分支的问题，可以先选择一个分支进行搜索。当搜索到某一结点，发现无法再继续搜索下去时，可以沿搜索路径回退到前一结点，沿另一分支继续搜索
- 如果回退之后没有其他选择，再沿搜索路径回退到更前结点，…。依次执行，直到搜索到问题的解，或搜索完全部可搜索的分支没有解存在为止
- 回溯法与分治法本质相同，可用递归算法求解

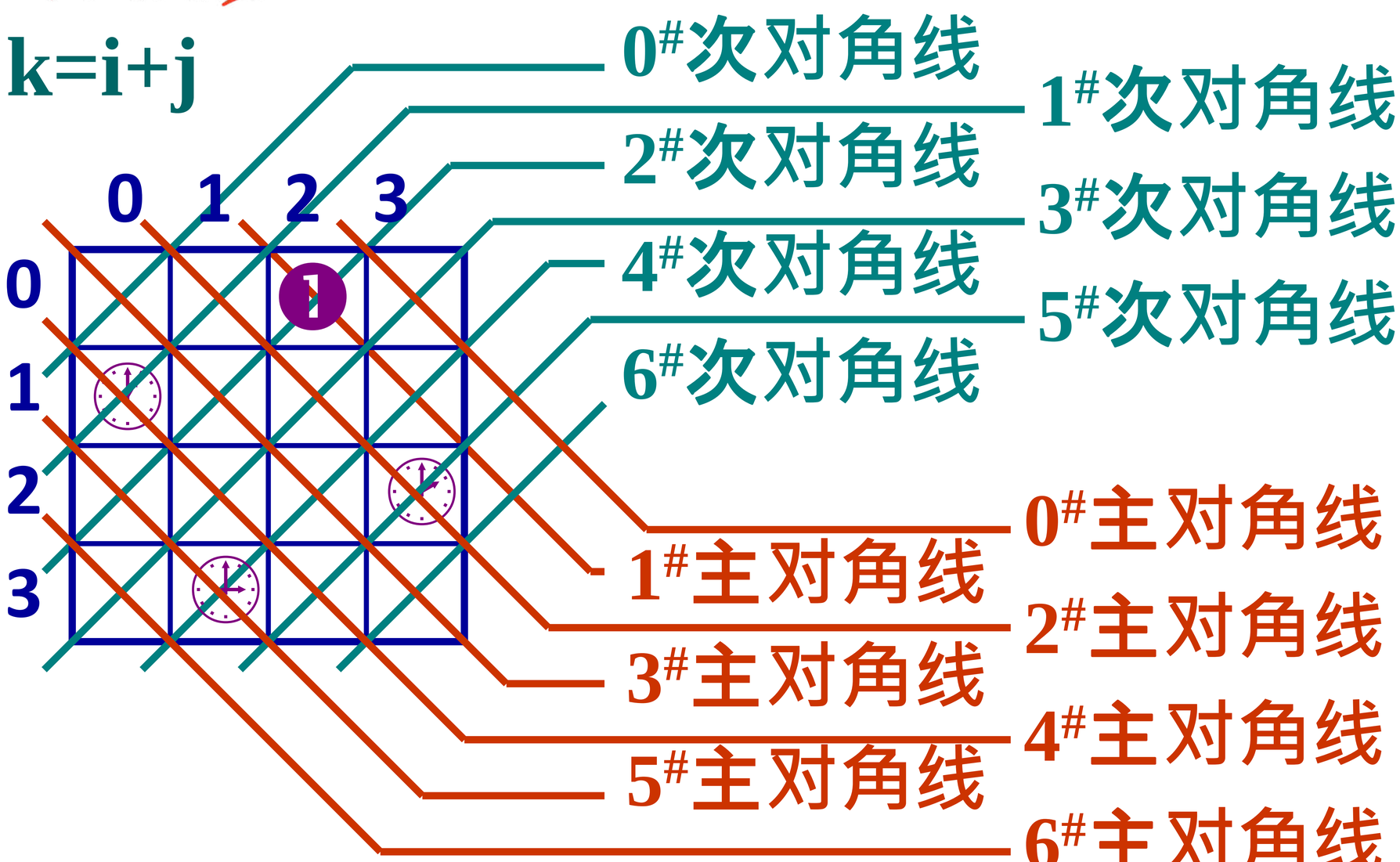
n皇后问题

- 在n行n列的国际象棋棋盘上，若两个皇后位于同一行、同一列、同一对角线上，则称为它们为互相攻击。n皇后问题是指找到这n个皇后的互不攻击的布局

n皇后问题

i+j为定值

k=i+j



k=n+i-j-1

i-j为定值

n皇后问题解题思路

- 安放第i行皇后时，需要在列的方向从0到n-1试探
- 在第j列试着安放一个皇后时
  - 若在列、主对角线、次对角线方向有其它皇后，则出现攻击，撤消在第j列安放的皇后
  - 若没有出现攻击，在第j列安放的皇后不动，递归安放第i+1行皇后

- 设置4个数组
  - col[n]：col[i]标识第i列是否安放了皇后
  - md[2n-1]：md[k]标识第k条主对角线是否安放了皇后
  - sd[2n-1]：sd[k]标识第k条次对角线是否安放了皇后
  - q[n]：q[i]记录第i行皇后在第几列

n皇后问题的递归算法

```
void Queen(int i) {
    for (int j=0;  j<n;  j++) {
        if (!col[j] && !md[n+i-j-1] && !sd[i+j]) {
            // 第i行第j列没有攻击，在第i行第j列安放皇后
            col[j]=md[n+i-j-1]=sd[i+j]=1;  q[i]=j;
            if (i==n-1) {  // 输出一个布局
                for (k=0; k<n; k++)  cout<<q[k]<<‘,’;
                cout<<endl;
            } else Queen(i+1);//递归安放下一行
            col[j]=md[n+i-j-1]=sd[i+j]=0;  q[i]=0; // 撤消第i行第j列的皇后
        }
    }
}
```

找到一种解决方案

为了找到所有的解决方案

练习1

- 用递归求两个正整数x和y的最大公约数
  - 如果x/y余数为0，则x和y的最大公约数是y
  - 否则，答案就是y和x/y的余数的最大公约数

```
int gcd(int x, int y) {
    if(x%y == 0) return y;
    else return gcd(y, x%y);
}
```

练习1

- 用递归求已知十进制整数的反向十进制整数
- 例如，对于1234，若已反向部分为43，则12还未被反向，则需要43*10 + 12%10 = 432，剩下12/10=1还未被反向

```
int reverseInt(int n, int s) {
    if(n<10) return s*10 + n;
    return reverseInt(n/10, s*10 + n%10);
}
```

05 递归和广义表

- 递归

- 递归转化为非递归

- 广义表

递归过程

- 递归过程在实现时，需要自己调用自己
- 层层向下递归，退出时的次序正好相反



- 主程序第一次调用递归过程为**外部调用**
- 递归过程每次递归调用自己为**内部调用**
- 每次递归返回时的**地址**是不同的

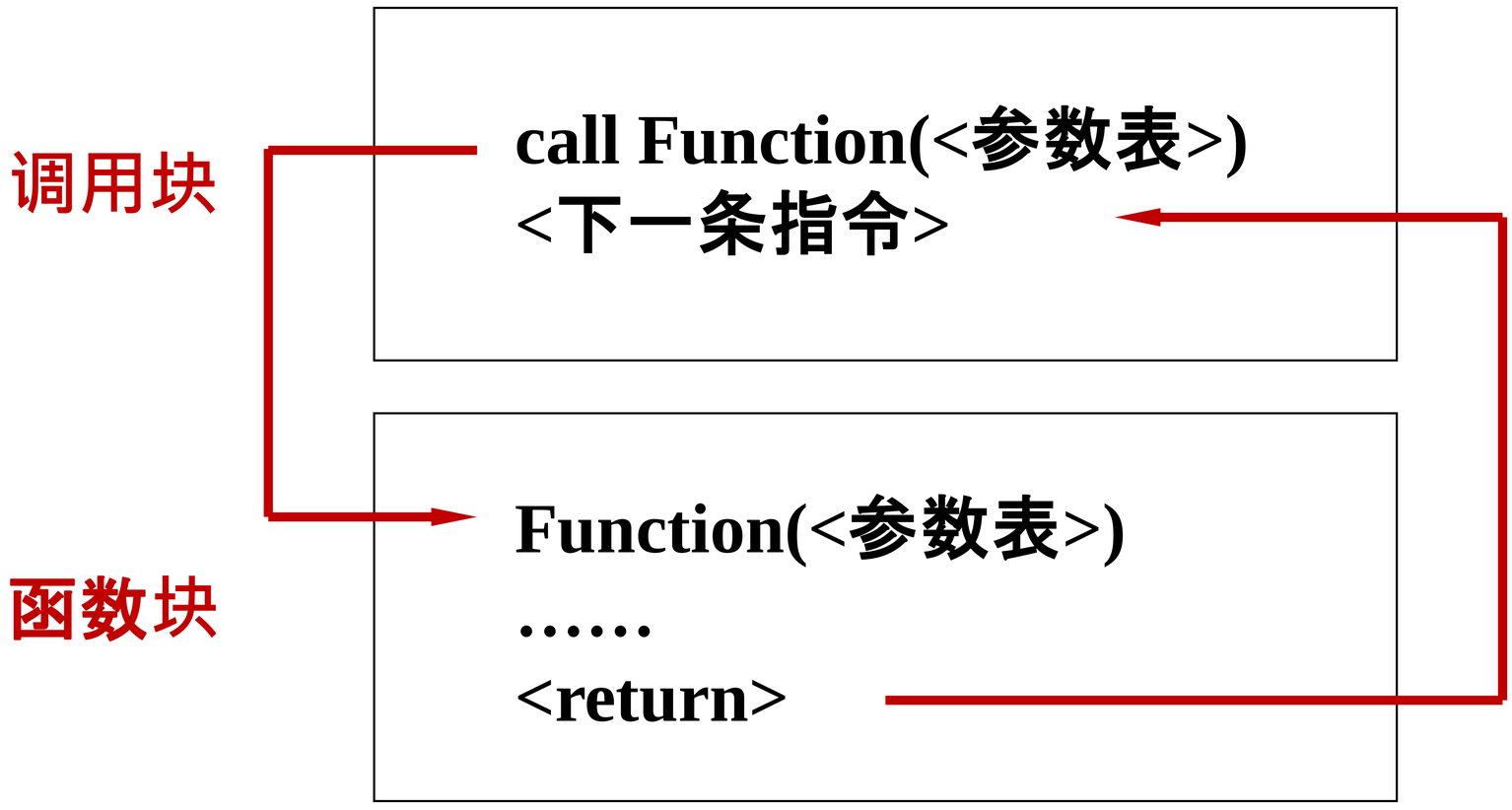
递归工作栈

- 每一次递归调用时，需要保存过程中的参数、局部变量、返回地址等，需要额外的存储空间，形成递归工作记录
- 在逐层递归的过程中，递归工作记录一般保存在栈中，这样栈的后进先出特征正好匹配递归层层返回的顺序

参数	局部变量	返回地址

递归工作记录

函数递归时的工作记录


调用块
call Function(<参数表>)
<下一条指令>
函数块
Function(<参数表>)
……
<return>
参数
局部变量
返回地址

返回地址

```
void main ( ) {
    int n;
    n = Fact (4);
}
```

RetLoc1

```
long Fact ( long n ) {
    if ( n == 0 ) return 1;
    else return n * Fact (n-1);
}
```

RetLoc2

计算Fact时活动记录的内容

0	1	RetLoc2	return 1 //返回 1
1	1	RetLoc2	return 1*1 //返回 1
2	2	RetLoc2	return 2*1 //返回 2
3	6	RetLoc2	return 3*2 //返回 6
4	24	RetLoc1	return 4*6 //返回 24
参数	局部变量	返回地址	返回时的指令

递归过程转化为非递归

- 递归方法是一种很有效很自然的分析和描述问题的方法
  - 递归过程简洁、易编、易懂，但递归过程**效率低**，**重复计算多**
  - 有时先用递归的思想分析和描述问题，然后转化成非递归的算法
  - 改为非递归过程的目的是**提高效率**

- 基本思路可以分为两个大的方向
  - 一是寻找与递归算法思路完全不同的非递归算法，有时比较困难
  - 二是利用递归的思路，但是进行改造，即把每次递归调用替换为入栈临时状态，每次递归返回替换为出栈临时状态，就可以构造出对应的非递归算法；在此基础上，进一步消除其中的冗余
  - 有些特殊递归有更简单的方法，如尾递归、线性递归等

斐波那契数列

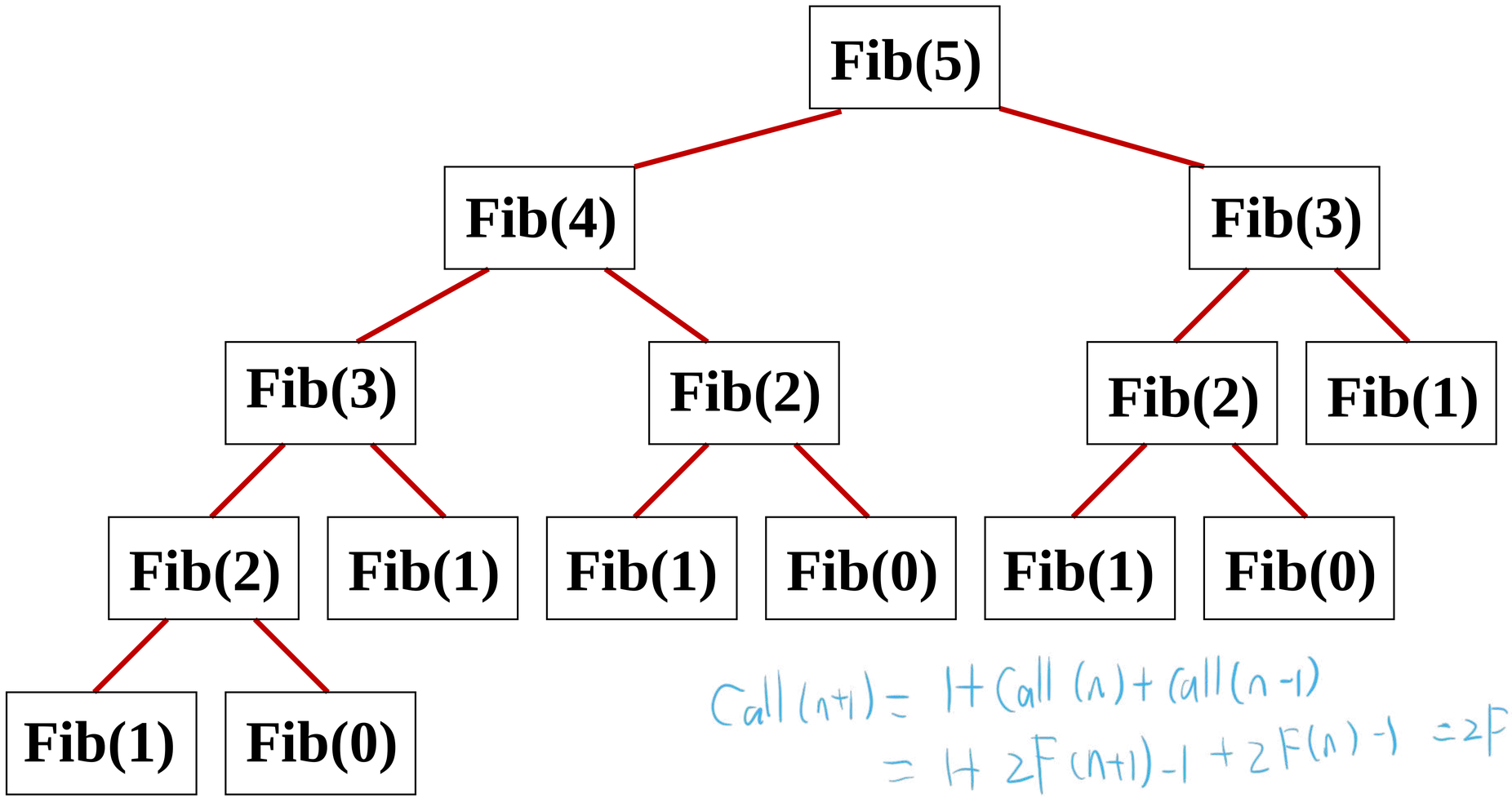
$$Fib(n)=\begin{cases} n, & n=0,1 \\ Fib(n-1)+Fib(n-2), & n>1 \end{cases}$$

- **Fib(0)=0, Fib(1)=1, Fib(2)=1, Fib(3)=2, Fib(4)=3, Fib(5)=5**

- **求解斐波那契数列的递归算法**

```
long Fib(long n) {
    if (n<=1)  return n;
    else return Fib(n-1) + Fib(n-2);
}
```

斐波那契数列的递归调用树

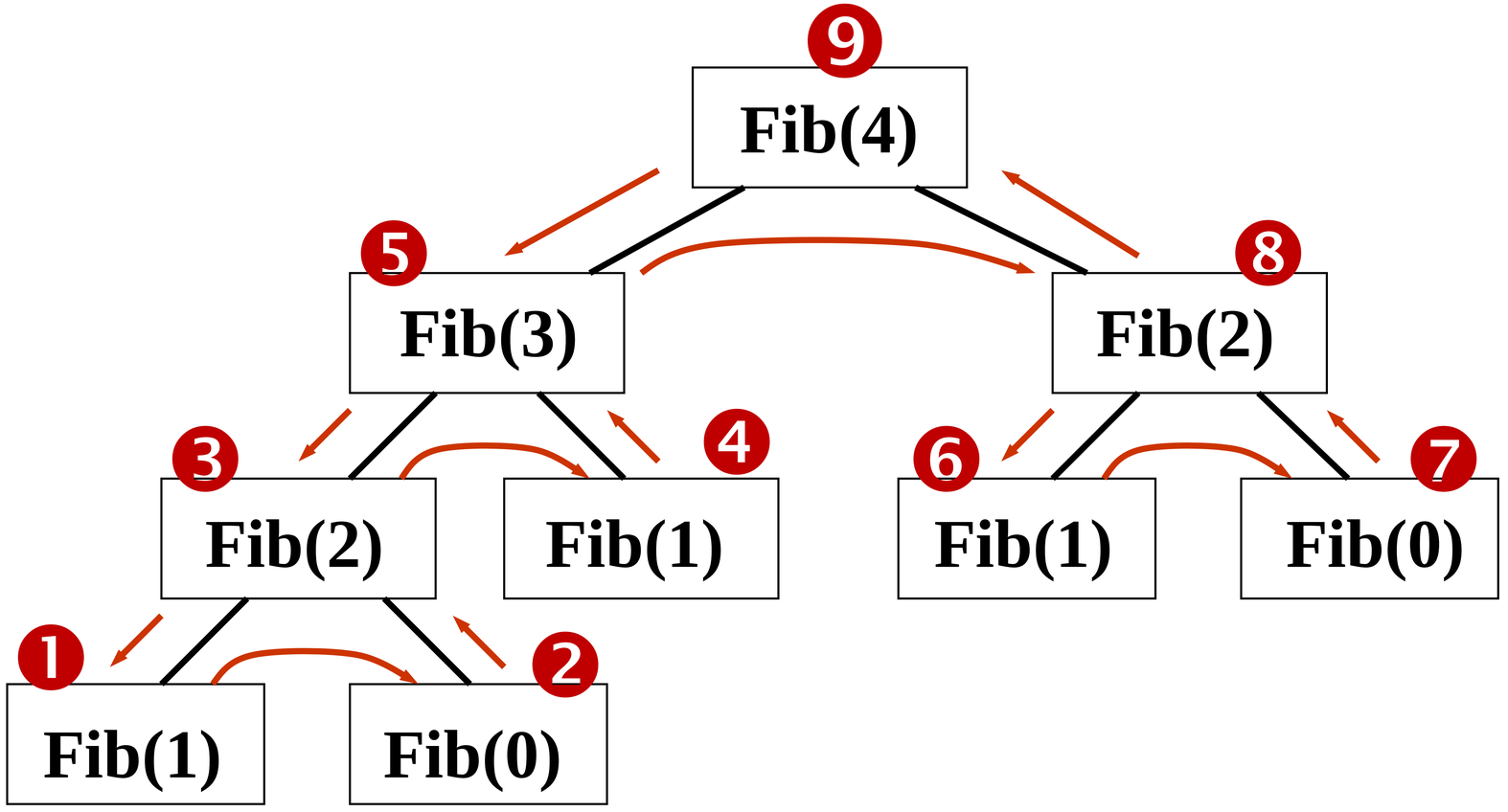


$Call(n+1) = 1 + Call(n) + call(n-1)$

$= 1 + 2F(n+1) - 1 + 2F(n) - 1 = 2F(n+2) - 1$

可用归纳法证明：调用次数NumCall(n) = 2*Fib(n+1) - 1

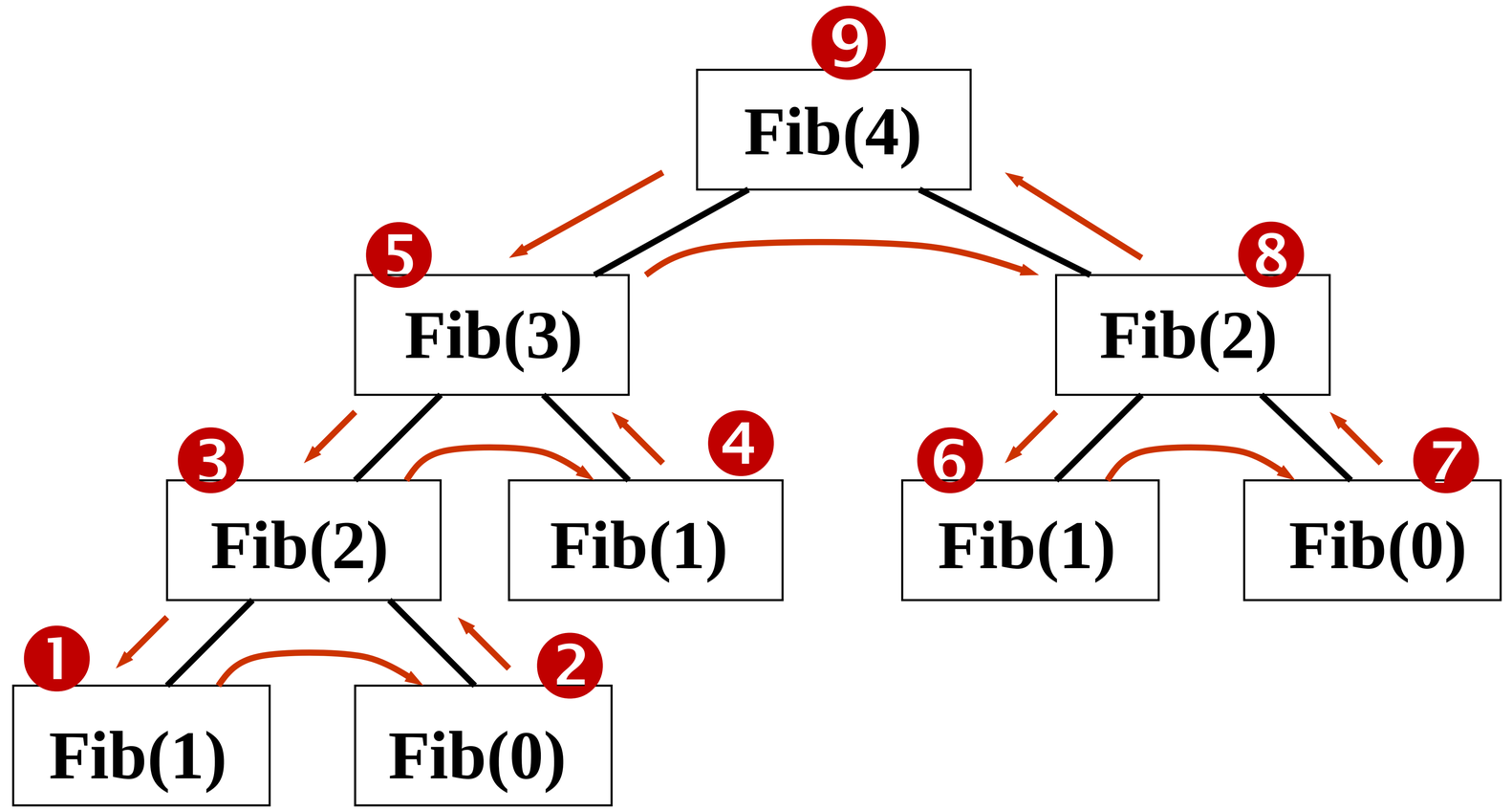
斐波那契数列的递归过程

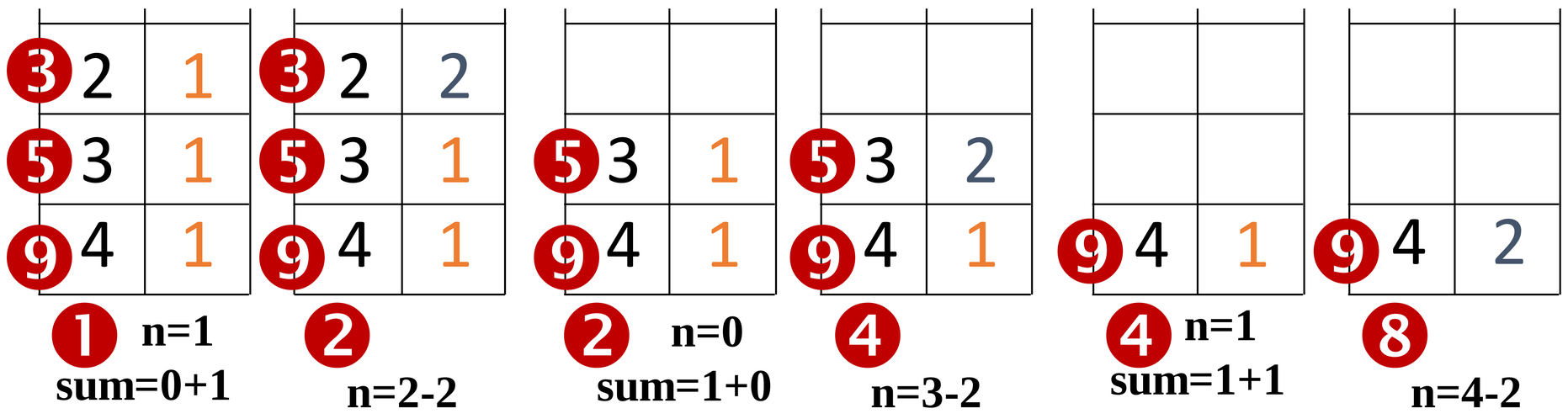


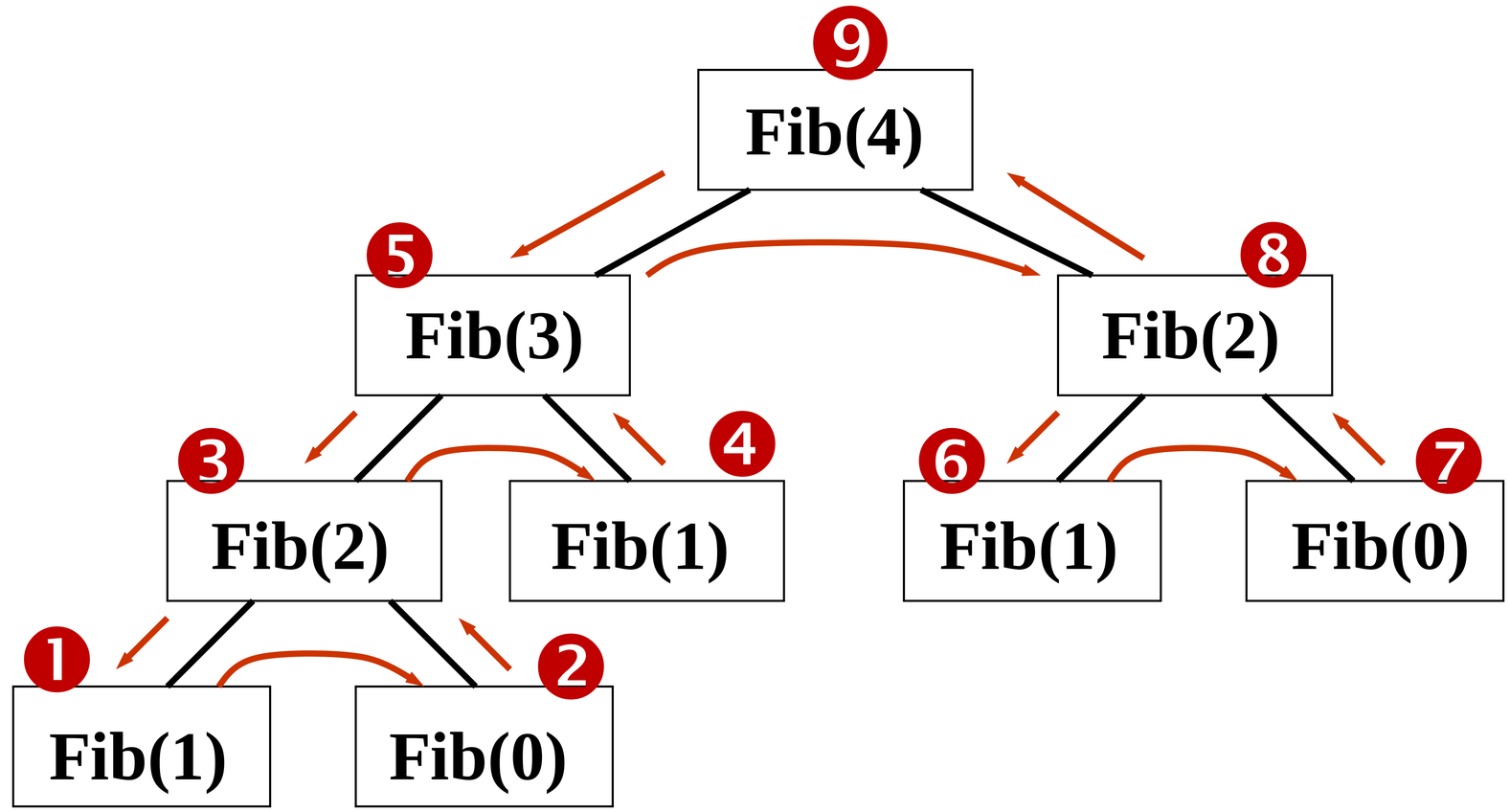
栈结点

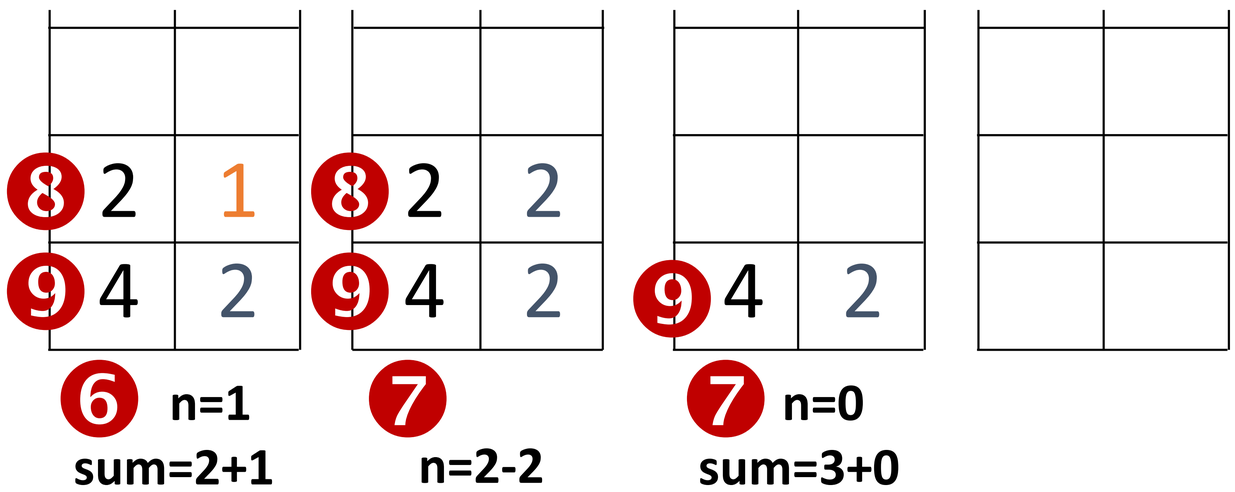
n	tag

tag = 1, 向左递归；tag = 2, 向右递归


9
Fib(4)
5
Fib(3)
8
Fib(2)
3
Fib(2)
4
Fib(1)
6
Fib(1)
7
Fib(0)
1
Fib(1)
2
Fib(0)


3 2 1
5 3 1
9 4 1
1 n=1
sum=0+1
3 2 2
5 3 1
9 4 1
2 n=2-2
5 3 1
9 4 1
2 n=0
sum=1+0
5 3 2
9 4 1
4 n=3-2
9 4 1
4 n=1
sum=1+1
9 4 2
8 n=4-2


9
Fib(4)
5
Fib(3)
8
Fib(2)
3
Fib(2)
4
Fib(1)
6
Fib(1)
7
Fib(0)
1
Fib(1)
2
Fib(0)


8 2 1
9 4 2
6 n=1
sum=2+1
8 2 2
9 4 2
7 n=2-2
9 4 2
7 n=0
sum=3+0

斐波那契数列的非递归算法

```
long Fibnacci(long n) {
    Stack <Node> S;  Node w;  long sum=0;
    // 反复执行直到所有终端结点数据累加完
    do {
        // 向左递归到底，边走边进栈
        while (n>1) { w.n=n;  w.tag=1;  S.Push(w);  n--; }
        sum=sum+n;  // 执行求和，基础情况
        while (!S.IsEmpty( )) {
            S.Pop(w);
            // 改为向右递归，F(n)右侧为F(n-2)
            if (w.tag==1) { w.tag=2;  S.Push(w);  n=w.n-2;  break; }
        }
    } while (!S.IsEmpty( ));
    return sum;
}
```

尾递归转为非递归

- 如果递归函数中只有一次递归调用，并且该调用是递归函数返回前的最后一条语句，这种情况称为尾递归
- 通常可以直接转化为循环

```
void recfunc(int A[ ],  int n) {
   if (n>=0) {
      cout<<A[n]<<“ ”;
      n--;
      recfunc(A, n);
   }
}
```



```
void sterfunc(int A[ ], int n) {
   while (n>=0) {
      cout<<“value ”<<A[n]<<endl;
      n--;
   }
}
```

线性递归转为非递归

- **比尾递归复杂一些的是线性递归，有以下的一般形式**

```
proc(int n){
    Stms1(n);
    if (Eval(n)) {
        proc(n-1);
        Stms2(n);
    }
    else Stms3(n);
}
```



```
proc(int n) {
    int k = n+1;
    do {
        k--;
        Stms1(k);
    } while Eval(k);
    Stms3(k);
    while (k<n) {
        k++;
        Stms2(k);
    }
}
```

线性递归转为非递归

- 对于求阶乘的问题，如果要考虑得更完善，需要防止在计算过程中发生溢出等异常情况

```
void factorial ( int n, int &temp ) {
    if ( n < 0 ) { cerr << n << "不能为负数" << endl;  exit(1); }
    if ( n > 0 ) {
        factorial ( n-1, temp );
        if (Max / temp > n) { temp = temp * n; }  // Max为最大整数值
        else { cerr << "计算结果溢出" << endl;  exit(1); }
    }
    else temp = 1;
}
```

线性递归转为非递归

```
void factorial ( int n, int &temp ) {
    int k = n + 1;
    do {
        k--;
        if ( k < 0 ) { cerr << k << "不能为负数" << endl;  exit(1); }
    } while ( k > 0 ) ;
    temp=1;
    while( k < n ) {
        k++;
        if (Max / temp > k) temp = temp * k;
        else { cerr << "计算结果溢出" << endl;  exit(1); }
    }
}
```

单向递归转为非递归

- 比线性递归更复杂一些的是单向递归，尾递归可以看作一种特殊的单向递归
  - 可以有多个递归调用，但都必须处于递归调用过程的最后
  - 要求递归过程中的这些递归调用的参数由主调用过程的参数决定，而处在同一递归层次的递归调用之间参数不会互相影响

- 单向递归同样可以较为方便地转化为循环实现
  - 由于单向递归中的递归调用处于递归过程的最后，因此同样可以从递归的终止条件开始，通过循环逐渐构造出整个问题的解，每一次循环完成的工作相当于从一个递归层次中返回
  - 用变量保存中间结果，不需要栈

单向递归转为非递归

```
long FibIter(long n) {
    if (n<=1)  return n;
    long twoback=0, oneback=1, current;
    for (int i=2; i<=n; i++) {
        current = twoback + oneback;
        twoback = oneback;
        oneback = current;
    }
    return  current;
}
```

05 递归和广义表

- 递归
- 递归转化为非递归
- 广义表

广义表(Generalized List)

- 广义表是线性表的推广，定义为n(≥0)个表元素组成的有限序列，记作$LS=(a_0, a_1, a_2, \ldots, a_{n-1})$
- LS是表名，a_i是表元素，它可以是表(称为子表)，也可以是数据元素(称为原子)
- n为表的长度，n=0的广义表为空表
- n>0时，第一个表元素称为广义表的表头(Head)，除此之外，其它表元素组成的表称为广义表的表尾(Tail)，即 $Head(LS)=a_0$，$Tail(LS)=(a_1, a_2, \ldots, a_{n-1})$

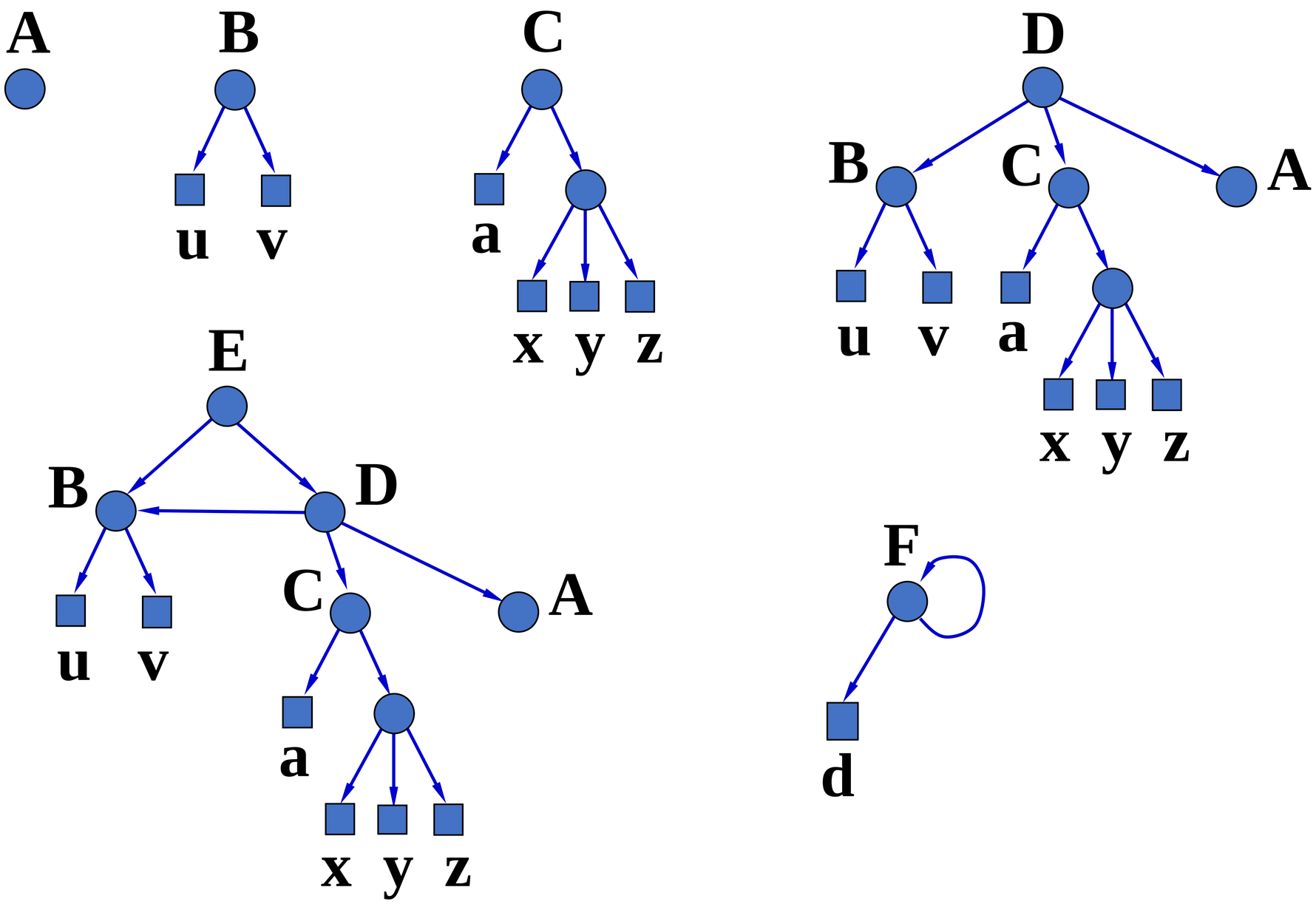
广义表的一些例子

- **A=( )**
  - **空表，长度为0，无表头和表尾**
- **B=(6, 2)**
  - **线性表(只有原子)，长度为2，表头为6，表尾为(2)**
- **C=(‘a’,(5, 3, ‘x’))**
  - **长度为2，表头为‘a’，表尾为((5, 3, ‘x’))**
- **D=(B, C, A)**
  - **长度为3，表头为B，表尾为(C, A)**
- **E=(B, D)**
  - **长度为2，表头为B，表尾为(D)**
- **F=(4, F)**
  - **长度为2，表头为4，表尾为(F)**

广义表的特性

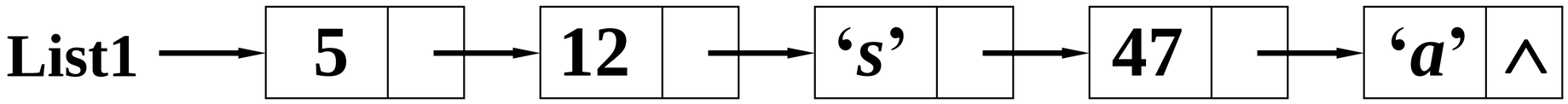
- 有次序性
  - 在广义表中，各表元素在表中以线性序列排列，每个元素至多一个直接前驱，一个直接后继，且次序不能交换
- 有长度
  - 广义表中元素个数一定，不能是无限的，可以是空表
- 有深度
  - 广义表是多层次结构，表元素可以是子表，子表的表元素还可以是子表
- 可递归
  - 广义表可以是自己的子表(如前页的F)
- 可共享
  - 广义表可以被其它广义表共享——共享表(如前页B由D和E共享)

广义表各元素间的层次关系

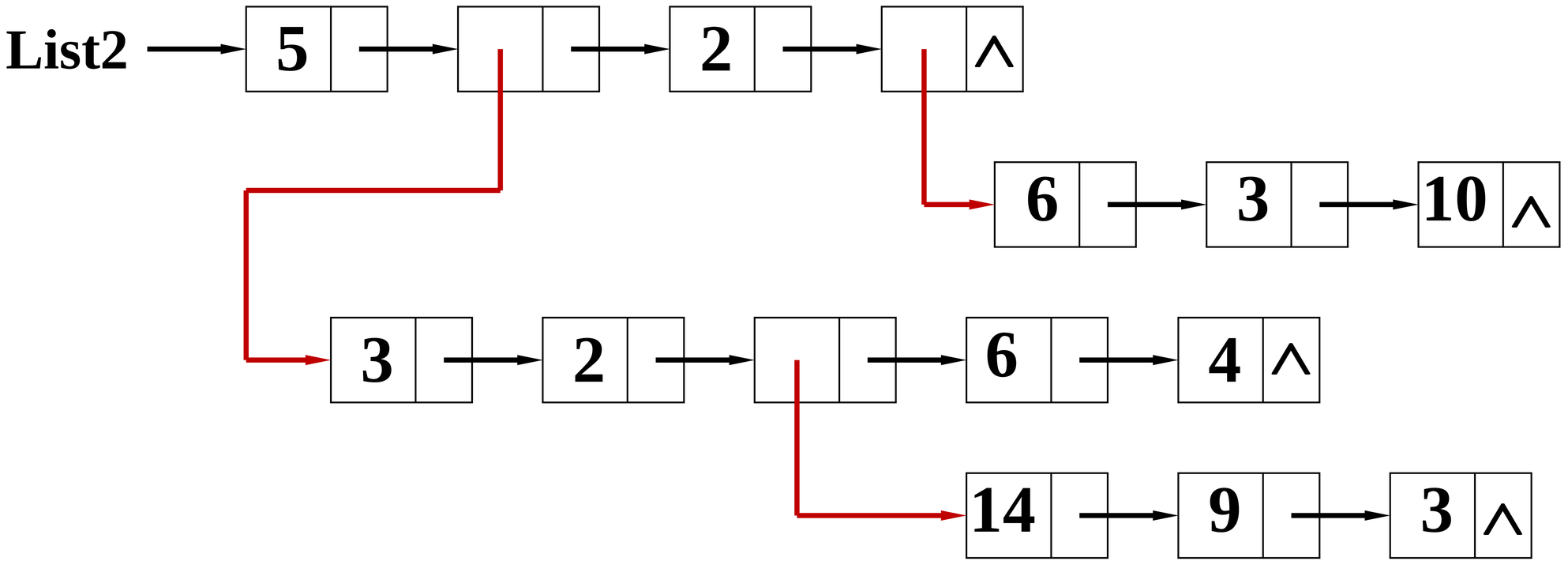


广义表的表示

- 只包括整数和字符型数据的广义表链表表示



- 表中套表情形下的广义表链表表示



练习2

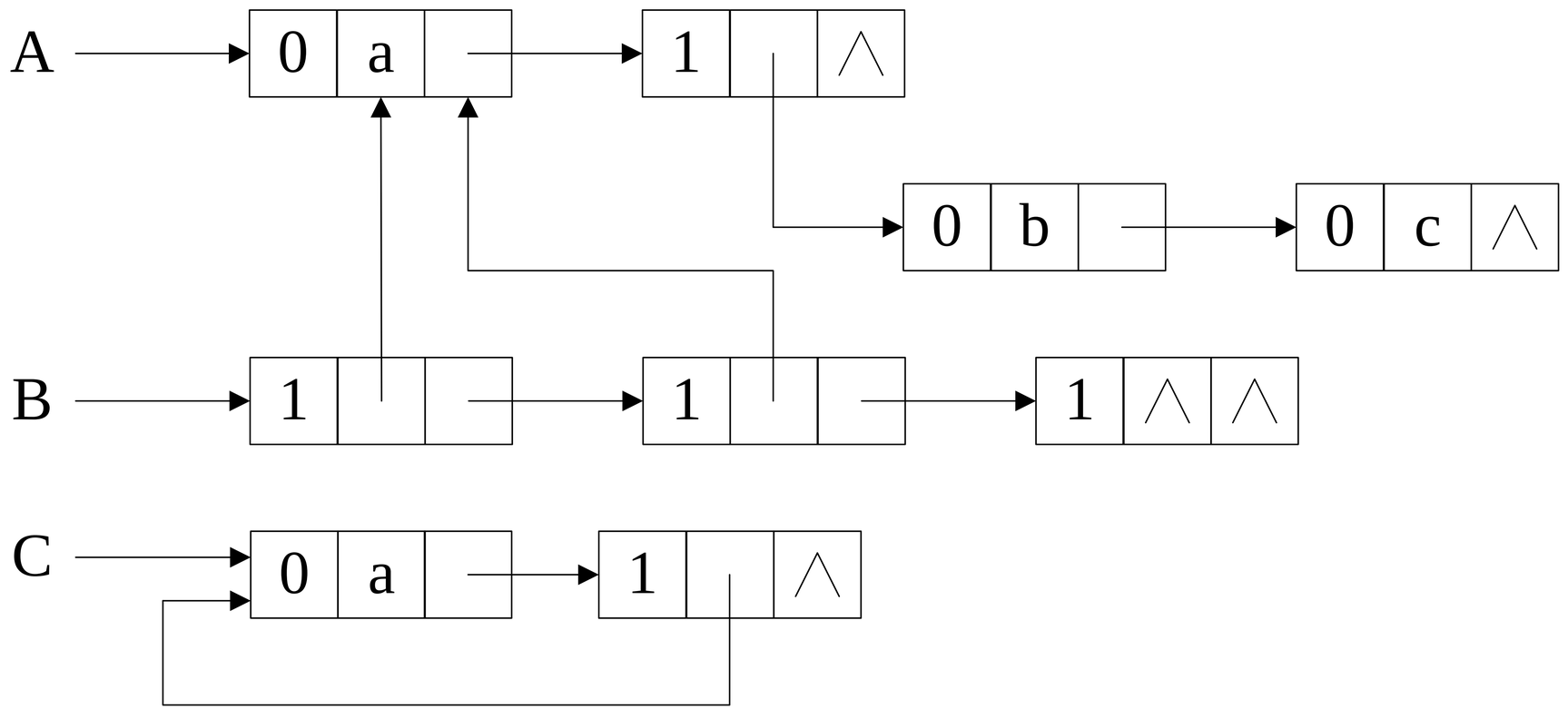
- 利用广义表的Head和Tail操作取出下列广义表中的banana元素
- L1=(apple, pear, banana, orange)
- Head(Tail(Tail(L1)))
- L2=((apple, pear), (banana, orange))
- Head(Head(Tail(L2)))
- L3=(apple, (pear, (banana), orange))
- Head(Head(Tail(Head(Tail(L3)))))

广义表结点定义(I)

tag	dlink/data	link

- 标志域**tag**：结点类型，tag=0，原子；tag=1，子表
- 信息域**info**：tag=0时，存放数据值(data)；tag=1时，存放指向子表表头的指针(dlink)
- 尾指针域**link**：指向同一层下一个结点

广义表结点定义(I)

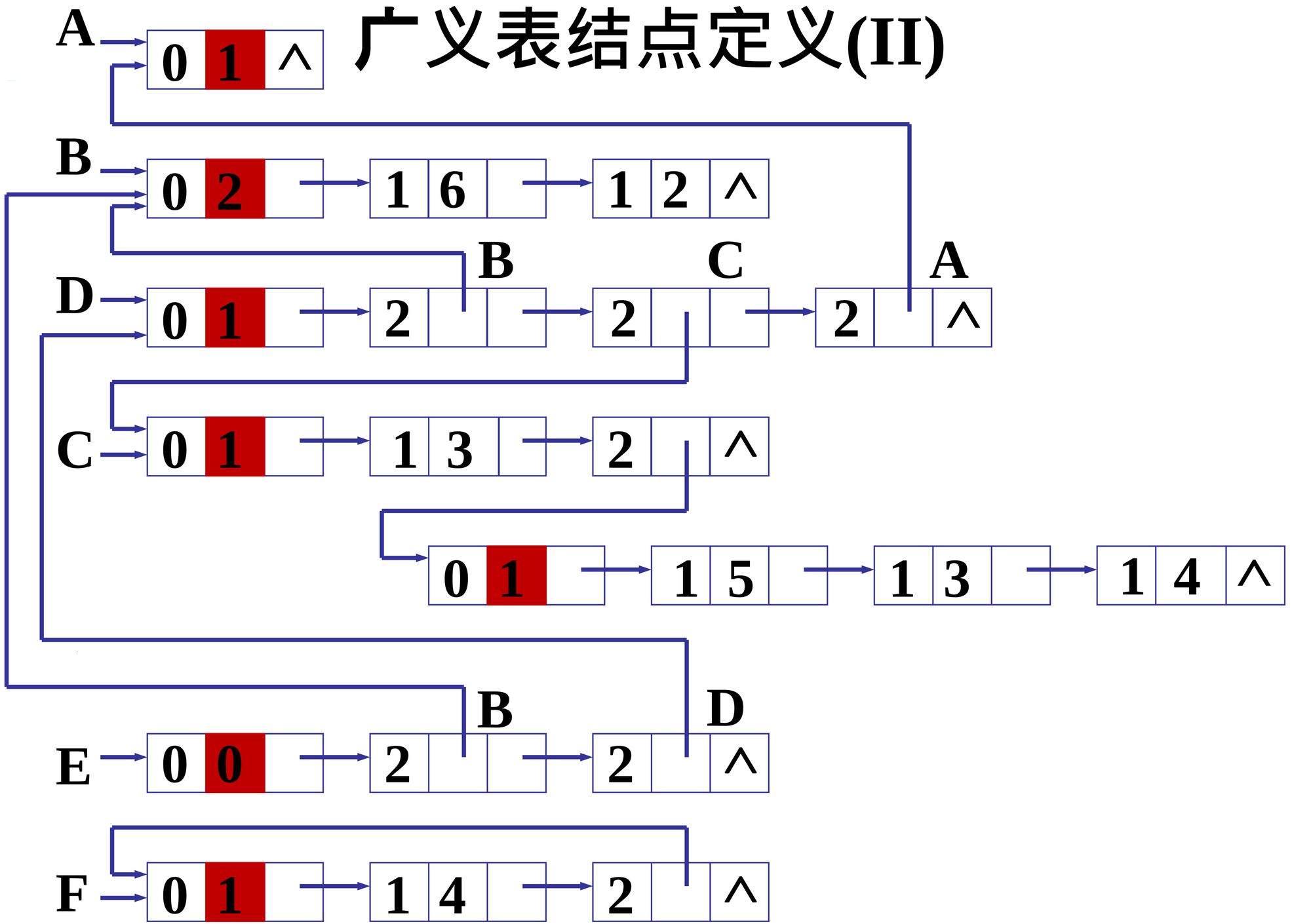


广义表结点定义(II)

utype	info	tlink

- 标志域**utype**：结点类型，utype=0，表头；utype=1，原子；utype= 2，子表
- 信息域**info**：utype=0时，存放引用计数(ref)；utype=1时，存放数据值(value)；utype=2时，存放指向子表表头的指针(hlink)
- 尾指针域**tlink**：utype=0时，指向该表第一个结点；utype!=0时，指向同一层下一个结点

广义表结点定义(II)



上述存储表示的特点

- 所有表(包括空表)，以及不论哪一层的子表，都带有一个表头结点，便于操作(特别是共享表情况下的删除操作)
- 表中结点的层次分明，所有位于同一层的表元素，在其存储表示中也在同一层
- 最高一层的表结点个数(除表头结点外)即为表长度

广义表的类定义

```
template <class Type> class GenList;
template <class Type> class GenListNode {  // 广义表结点类定义
    friend class Genlist;
    private:
        int utype;  // =0/1/2
        GenListNode <Type> *tlink;  // 指向同一层下一结点的指针
        union {  // 联合
            int ref;  // utype=0，存放引用计数
            Type value;  // utype=1，存放数值
            GenListNode <Type> *hlink;  // utype=2，存放指向子表的指针
        } info;
    public:
        GenListNode( ) : utype(0), tlink(NULL), info.ref(0) { }
        GenListNode(GenListNode <Type> &RL) {
            utype = RL.utype;  tlink = RL.tlink;  info=RL.info;  }
};
```

广义表的类定义

```
template <class Type> class GenList {  // 广义表类定义
  private:
    GenListNode <Type> * first;  // 广义表头指针
    // 复制一个ls指示的无共享非递归表
    GenListNode <Type> * Copy(GenListNode <Type> *ls);
    // 计算由ls指示的非递归表的深度
    int Depth(GenListNode <Type> *ls);
    // 计算由ls指示的非递归表的长度
    int Length(GenListNode <Type> *ls);
    // 比较以s和t为表头的两个表是否相等
    bool Equal(GenListNode <Type> *s, GenListNode <Type> *t);
    // 释放以ls为表头结点的广义表
    void Remove(GenListNode <Type> *ls);
```

广义表的类定义

```
public:
    // 构造函数
    GenList( ) { first = new GenListNode <Type>; assert( first!=NULL ); }
    ~GenList( );  // 析构函数
    bool Head(GenListNode<Type> &x);  // 返回表头元素
    bool Tail(GenList <Type> &lt);  // 返回表尾
    GenListNode <Type> * First( );  // 返回第一个元素
    // 返回表元素elem的直接后继元素
    GenListNode <Type> * Next(GenListNode <Type> *elem);
    void Copy(const GenList <Type> &R);  // 复制广义表
    int Length( );  // 计算广义表的长度
    int Depth( );  // 计算广义表的深度
    void DelValue(GenListNode <Type> *ls, Type x);  // 删除值为x的结
点
};
```

广义表类的访问成员函数

```
template <class Type> bool GenList <Type> ::
Head(GenListNode <Type> &x) {
    if (first->tlink==NULL)  return false;
    else {  // 非空表
        x.utype = first->tlink->utype;  x.info = first->tlink->info;
        return true;
    }
}
template <class Type> bool GenList <Type> ::
Tail(GenList <Type> &lt) {
    if (first->tlink==NULL)  return false;
    else {  // 非空表
        lt.first->utype = 0;  lt.first->info.ref = 0;  //加括号
        lt.first->tlink = Copy(first->tlink->tlink);
        return true;
    }
}
```

广义表类的访问成员函数

```
template <class Type> GenListNode <Type> * GenList <Type> ::
First( ) {
    if (first->tlink==NULL)  return NULL;
    else return first->tlink;
}
```

```
// 返回表元素elem的直接后继元素
template <class Type> GenListNode <Type> * GenList <Type> ::
Next(GenListNode <Type> *elem) {
    if (elem->tlink==NULL)  return NULL;
    else return elem->tlink;
}
```

广义表的递归算法

- 广义表的结构可以用递归的方式进行描述，因此很多广义表的算法也可以表达成递归算法
  - 复制广义表
  - 计算广义表的长度
  - 计算广义表的深度
  - 删除广义表中所有含x的结点
  - 析构广义表
  - ……

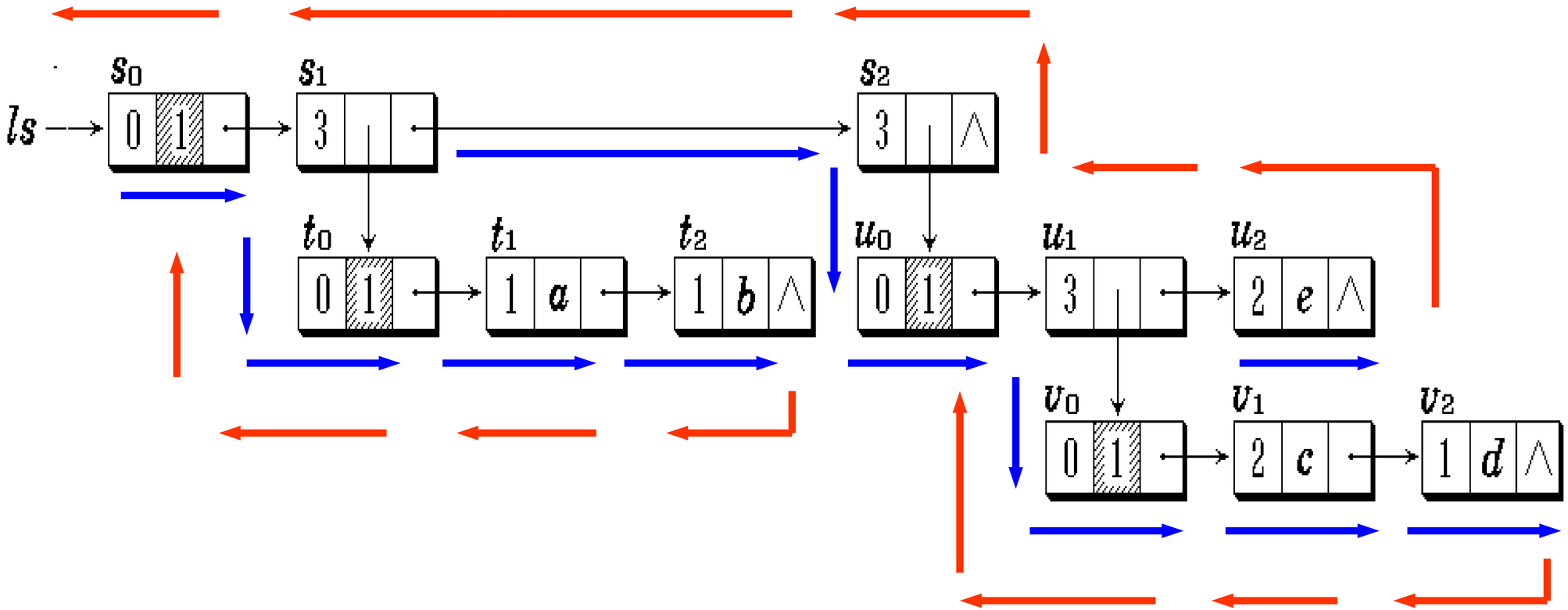
复制广义表

- **算法思路**
  - **从**给定的广义表的第一个结点一直复制到最后一个结点
- 递归过程
  - 递归终止条件：**当前复制的**结点为空
  - **否**则，**先复制当前**结点，**若当前**结点是子表结点，则递归复制其**引用的子表**；**再**递归复制当前结点的后继结点

复制广义表

```
template <class Type> void GenList <Type> ::
Copy(const GenList <Type> &R) {  // 公有函数
    first = Copy(R.first);
}
template <class Type> GenListNode * GenList <Type> ::
Copy(GenListNode <Type> *ls) {  // 私有函数
    GenListNode <Type> *q = NULL;
    if (ls!=NULL) {
        q = new GenListNode <Type>;  q->utype = ls->utype;
        switch (ls->utype) {
            case 0: q->info.ref = ls->info.ref;  break;
            case 1: q->info.value = ls->info.value;  break;
            case 2: q->info.hlink = Copy(ls->info.hlink);  break; }
        q->tlink = Copy(ls->tlink);
    }
    return q;
}
```

复制广义表



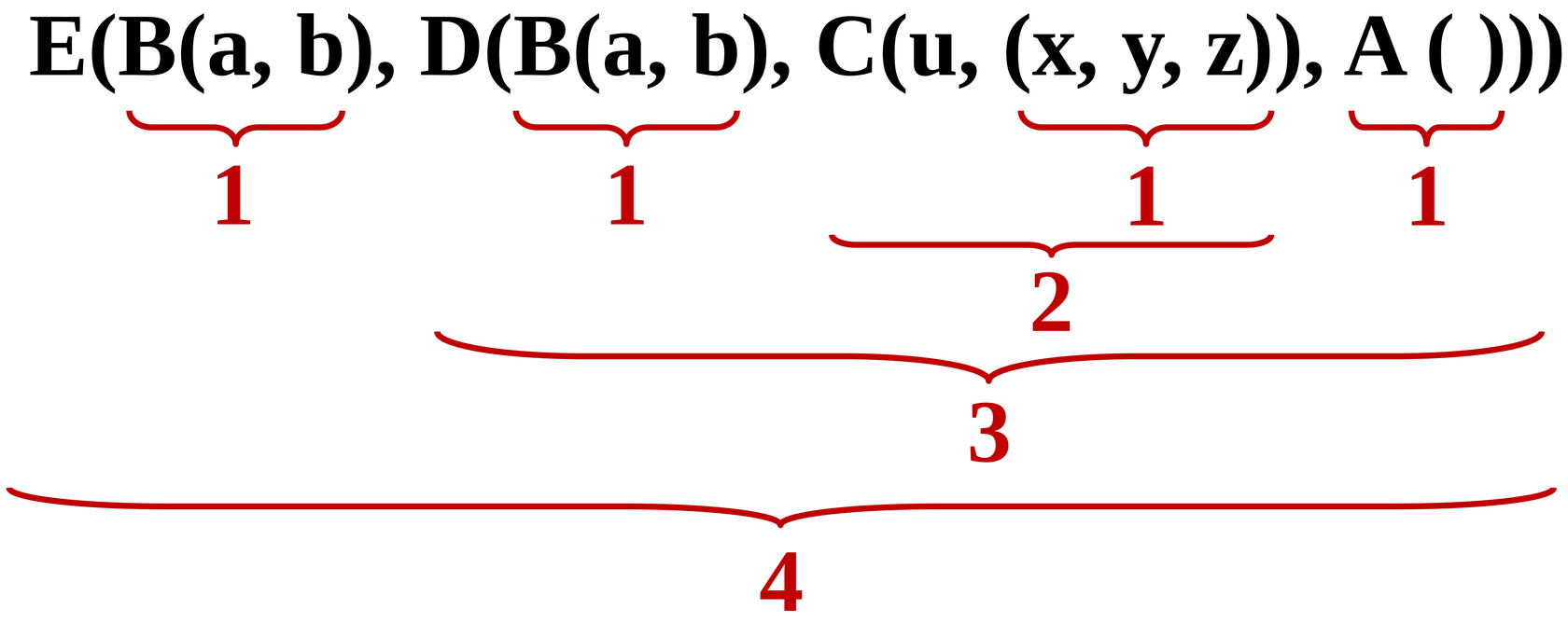
计算广义表的长度

```
template <class Type> int GenList <Type> ::
Length( ) {  // 公有函数
    return Length(first->tlink);
}

template <class Type> int GenList <Type> ::
Length(GenListNode <Type> *ls) {  // 私有函数
    if (ls!=NULL)  return 1+Length(ls->tlink);
    else return 0;
}
```

计算广义表的深度

$$\mathrm{Depth}(LS)=\begin{cases}1, & \text{当LS为空表时}\\ 0, & \text{当LS为原子时}\\ 1+\max\limits_{0\leqslant i\leqslant n-1}\{\mathrm{Depth}(a_i)\}, & \text{其他，}n\geqslant 1\end{cases}$$

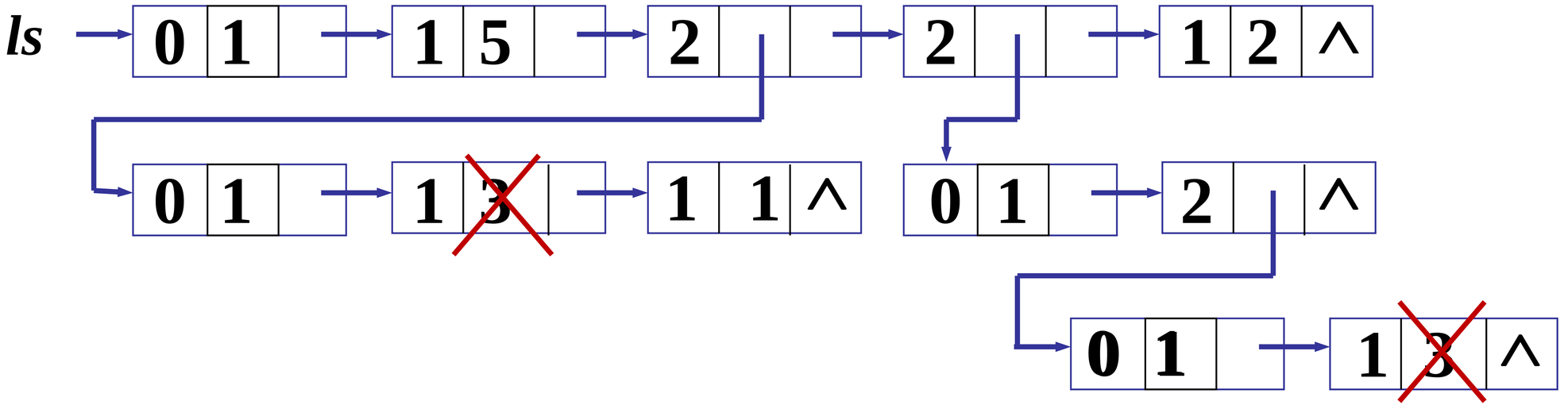


计算广义表的深度

```
template <class Type> int GenList <Type> :: Depth( ) {
    return Depth(first);
}

template <class Type> int GenList <Type> ::
Depth(GenListNode <Type> *ls) {
    if (ls->tlink==NULL)  return 1;  // 空表
    GenListNode <Type> *temp = ls->tlink;  int m = 0, n;
    while (temp!=NULL) {  // 在表顶层横扫
        if (temp->utype==2) {  // 结点为子表
            n = Depth(temp->info.hlink);
            if (m<n)  m = n;  // m为最大深度
        }
        temp = temp->tlink;
    }
    return m+1;
}
```

删除广义表中所有含x的结点



- 扫描子链表
  - 若结点数据为x，删除，可能做循环连续删
  - 若结点数据不为x，不执行删除
  - 若结点为子表，递归在子表执行删除

删除广义表中所有含x的结点

```
template <class Type> void GenList <Type> ::
DelValue(GenListNode <Type> *ls, Type x) {
    if (ls->tlink!=NULL) {  // 非空表
        GenListNode <Type> *p = ls->tlink;
        while (p!=NULL && (p->utype == 1 && p->info.value == x)) {
            ls->tlink = p->tlink;  delete p;  // 删除
            p = ls->tlink;  // 指向同一层后继结点
        }
        if (p!=NULL) {
            if (p->utype==2)  // 在子表中递归删除
                DelValue(p->info.hlink, x);
            DelValue(p, x);  // 在以p为表头的链表中递归删除 后继结点
        }
    }
}
```

析构广义表

- 删除一个广义表时，首先要看它是否为几个表共享
- 检查表头结点中的引用计数，如果有多个地方共享，则不能删除，否则可能会造成其它地方使用出错
- 删除时先把该引用计数减1，只有引用计数减到0时才能真正做删除

析构广义表

```
template <class Type> GenList <Type> :: ~GenList( ) { Remove(first); }
template <class Type> void GenList <Type> ::
Remove(GenListNode <type> *ls) {
    ls->info.ref--;  // 引用计数减一
    if (ls->info.ref <= 0) {  // 引用计数减至0才能删除
        GenListNode <Type> *q;
        while (ls->tlink!=NULL) {  // 横扫链表
            q = ls->tlink;
            if (q->utype == 2) {  // 递归删除子表
                Remove(q->info.hlink);
                if (q->info.hlink->info.ref <= 0)
                    delete q->info.hlink;  // 删除子表附加头结点
            }
            ls->tlink = q->tlink;  delete q;  //删除当前结点。
        }
    }
}
```

判断两个广义表是否相等

- 不但两个广义表具有相同的结构，而且对应的数据成员具有相等的值
  - 如果两个广义表都是空表，则相等
  - 如果两个广义表的对应结点都是原子结点，对应项的值相等，再递归比较同一层后面的表元素
  - 如果两个广义表中对应项是子表结点，则递归比较相应的子表

判断两个广义表是否相等

```
template <class Type> bool GenList <Type> ::
Equal(GenListNode <Type> *s, GenListNode <Type> *t) {
    int x;
    if (s->tlink==NULL && t->tlink==NULL)
        return true;  // 表s与表t都是空表或者所有结点都比较完
    if (s->tlink!=NULL && t->tlink!=NULL &&
        s->tlink->utype==t->tlink->utype) {  // 两表都非空且结点标志相同
        if (s->tlink->utype==1)  // 原子结点，比较对应数据
            x = s->tlink->info.value==t->tlink->info.value ? 1 : 0;
        else if (s->tlink->utype==2)  // 子表结点，递归比较子表
            x = Equal(s->tlink->info.hlink, t->tlink->info.hlink);
        if (x==1)  return Equal(s->tlink, t->tlink);  // 递归比较同一层下一结点
    }
    return false;
}
```

练习3

- 若广义表满足Head(A) = Tail(A)，则A为(())，其长度和深度分别是多少？
- 长度为1，深度为2

练习3

- 广义表具有可共享性，因此在遍历一个广义表时必须为每一个结点增加一个标志域mark，以记录该结点是否访问过。一旦作了mark，以后就不再访问该结点。试采用递归的算法对一个非递归的广义表进行遍历

练习3

```
class GenList;
class GenListNode {
    friend class GenList;
    private:
        int mark;  // mark是访问标记
        int utype;  // utype=0/1/2
        GenListNode *tlink;  // 指向同层下一结点
        union{
            char listname;  // utype=0，表头结点存表名
            char atom;  // utype=1，存原子结点数据
            GenListNode* hlink; // utype=2，指向子表
        }value;
    public:
        ……
};
```

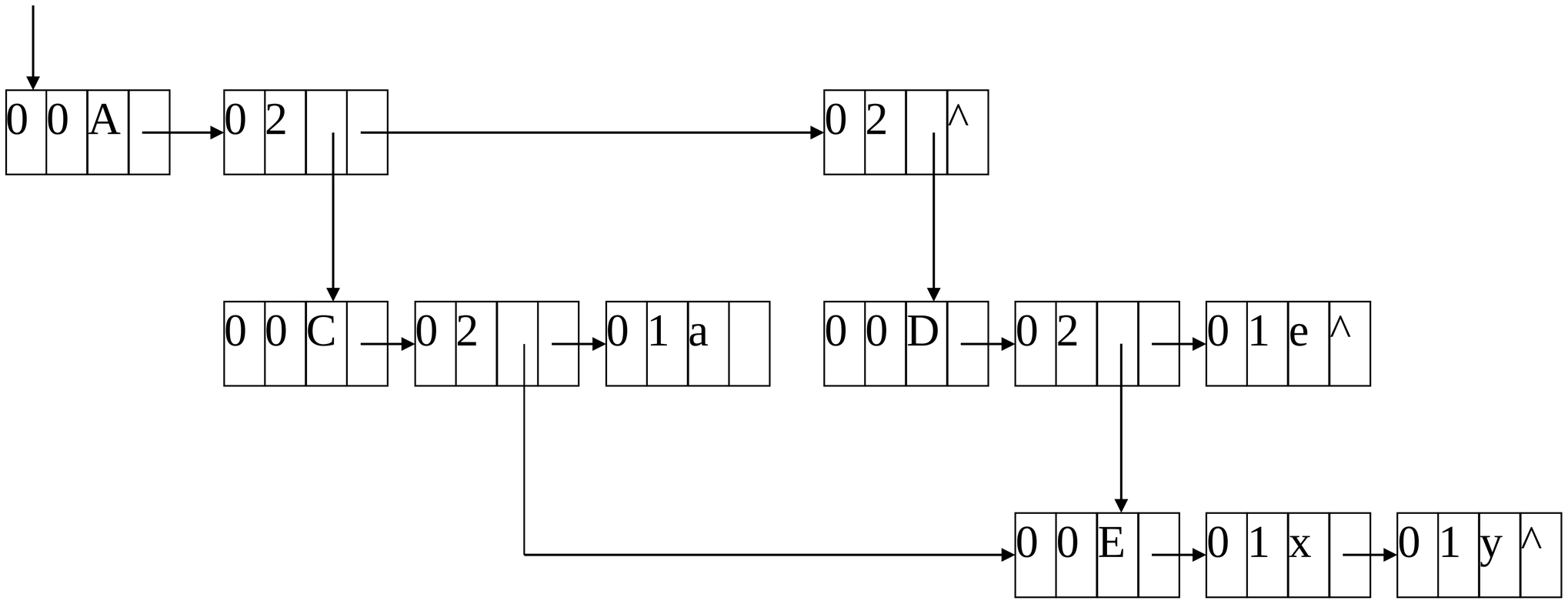
练习3

```
class GenList {  // 广义表类定义
    private:
        GenListNode *first;
        void traverse(GenListNode * ls);  // 遍历广义表
        void Remove(GenListNode * ls);
        ……
    public:
        GenList( );
        ~GenList( );
        void traverse( );  // 遍历广义表
        ……
}
```

练习3

```
void GenList::traverse() { traverse(first); }
void GenList::traverse(GenListNode *ls) {
    if (ls!=NULL) {
        ls->mark = 1;
        if (ls->utype == 0) cout<<ls->value.listname<<'('; // 表头结点
        else if (ls->utype==1) {  // 原子结点
            cout<<ls->value.atom;
            if (ls->tlink!=NULL) cout<<',';
        } else if (ls->utype==2) {  // 子表结点
            if (ls->value.hlink->mark==0)  traverse(ls->value.hlink);  // 递归子表
            else cout<<ls->value.hlink->value.listname; //如果已遍历过，只输出表名
            if (ls->tlink!=NULL) cout<<',';
        }
        traverse(ls->tlink);  // 递归同一层下一节点
    } else cout<<')';
}
```

练习3



A(C(E(x, y), a), D(E, e))

本章小结

- 递归
- 递归转化为非递归
- 广义表

作业

- 实现迷宫问题的递归算法
- 实现Hanoi塔问题的非递归算法
- 实现n皇后问题的非递归算法
- 已知Ackerman函数定义，实现其递归与非递归算法

$$akm(m,n)=\begin{cases} n+1 & m=0 \\ akm(m-1,1) & m\neq 0, n=0 \\ akm(m-1,akm(m,n-1)) & m\neq 0, n\neq 0 \end{cases}$$

- 实现广义表遍历的非递归算法